

SMART STUDENT EXPENSE TRACKER

VITYARTHI – Build Your Own Project

PROJECT REPORT

*A Python-based modular application for recording,
managing and analyzing student expenses.*

SUBMITTED BY: AKSHAT SINGH

REGISTRATION NUMBER: 26BAI10594

SCHOOL: SCHOOL OF COMPUTING SCIENCE
ENGINEERING AND ARTIFICIAL INTELLIGENCE (SCAI)

COURSE: PYTHON PROGRAMMING

1. INTRODUCTION

Managing daily expenses is an important practical skill for college students. Small purchases such as food, transportation, educational materials and entertainment can accumulate quickly. However, students may not maintain a structured record of these transactions.

The Smart Student Expense Tracker is a menu-driven Python application designed to provide a simple local solution. It allows a student to enter expenses, view and delete records, analyze spending patterns, and generate a text report. The project demonstrates fundamental Python programming concepts in a real-world context.

2. PROBLEM STATEMENT

Students often make multiple small payments every day but lack a simple system for recording and analyzing them. As a result, they may not know their total expenditure, their highest spending category, or their largest individual expense.

Proposed solution: Develop a modular Python application that stores expense records locally, validates user input, performs calculations, and presents useful spending summaries.

3. OBJECTIVE

- Create a simple system for recording student expenses.
- Provide persistent storage using a CSV file.
- Allow users to view and delete expense records.
- Calculate total and average expenditure.
- Analyze spending category-wise.
- Identify the highest spending category and largest individual expense.
- Demonstrate modular Python programming, file handling, validation and unit testing.

4. FUNCTIONAL REQUIRMENTS

ID	Functional Requirement	Description
FR1	Student Input	Accept and validate student name and ID.
FR2	Add Expense	Accept date, category, description and amount.
FR3	View Expenses	Display all stored expense records.
FR4	Delete Expense	Delete a selected expense using its ID.
FR5	Analysis	Calculate totals, averages and category-wise spending.
FR6	Reporting	Generate a text summary report.
FR7	Validation	Handle invalid dates, amounts, categories and choices.

5. Non-Functional Requirements

Requirement	Description
Performance	Normal student-sized datasets should be processed quickly.
Usability	Menu-driven interface with clear prompts and messages.
Reliability	Expense records are stored persistently in CSV format.
Maintainability	Separate modules make the application easier to modify.
Error Handling	Invalid user input is handled without terminating the program.
Resource Efficiency	Uses local storage and Python standard-library features.

6. System Architecture

The system follows a modular architecture. The main controller communicates with functional modules, while the storage module manages the CSV file.

Architecture:

```
User / CLI → main.py → user.py / expenses.py / analysis.py →  
validation.py → storage.py → expenses.csv
```

The report.py module can use the analysis and storage modules to create a text report.

7. Workflow Diagram

Start → Enter Student Details → Display Menu → Select Operation → Validate Input →
Process Request

→ Save/Read Data → Display Result → Return to Menu → Exit

The workflow is designed as a continuous menu loop so the user can perform multiple operations during one session.

8. Use Case Diagram

The primary actor is the student. The student interacts with the system through the following use cases:

- Student → Add Expense
- Student → View Expenses
- Student → Delete Expense
- Student → Analyze Spending
- Student → Generate Report
- Student → Exit Application

9. Sequence Diagram

A typical Add Expense sequence is:

Student → main.py: Select Add Expense

main.py → expenses.py: Start expense

creation expenses.py → validation.py:

Validate date/category/amount validation.py

→ expenses.py: Return valid values

expenses.py → storage.py: Save record

storage.py → CSV: Write record expenses.py

→ main.py: Return success main.py →

Student: Display confirmation

10. Component / Class Design

Module	Responsibility
main.py	Controls menu and application flow.
user.py	Collects and validates student details.
expenses.py	Implements add, view and delete operations.
validation.py	Validates dates, amounts and categories.
storage.py	Creates, reads and writes the CSV data file.
analysis.py	Calculates totals and spending statistics.
report.py	Generates a text report.
config.py	Stores application constants.
tests/	Contains automated unit tests.

11. Database / Storage Design

A CSV file is used as lightweight persistent storage. Each row represents one expense.

Expense schema:

Field	Type	Purpose
id	Integer	Unique expense identifier.
date	String (YYYY-MM-DD)	Date of transaction.
category	String	Expense category.
description	String	Short description.
amount	Float	Expense amount in rupees.

12. Design Decisions & Rationale

- **Python:** Selected because it directly supports the programming concepts covered in the course and enables rapid development.
- **CSV storage:** Chosen because it is simple, portable, human-readable and does not require an external database server.
- **Modular structure:** Functions are divided across modules to improve readability, testing and maintainability.
- **Input validation:** Prevents common runtime errors caused by invalid dates, amounts and menu selections.
- **Unit testing:** Provides automated verification for important calculations and configuration.

13. Implementation Details

The application uses functions, conditional statements, loops, lists, dictionaries, exception handling, the csv module, pathlib, and unittest. The main program repeatedly displays a menu and calls the appropriate module based on the user's choice.

Expense records are represented as dictionaries and stored as CSV rows. The analysis module converts stored amounts to numeric values and calculates total expenditure, average expenditure and category totals.

14. Project File Structure

```
Student_Expense_Tracker/
```

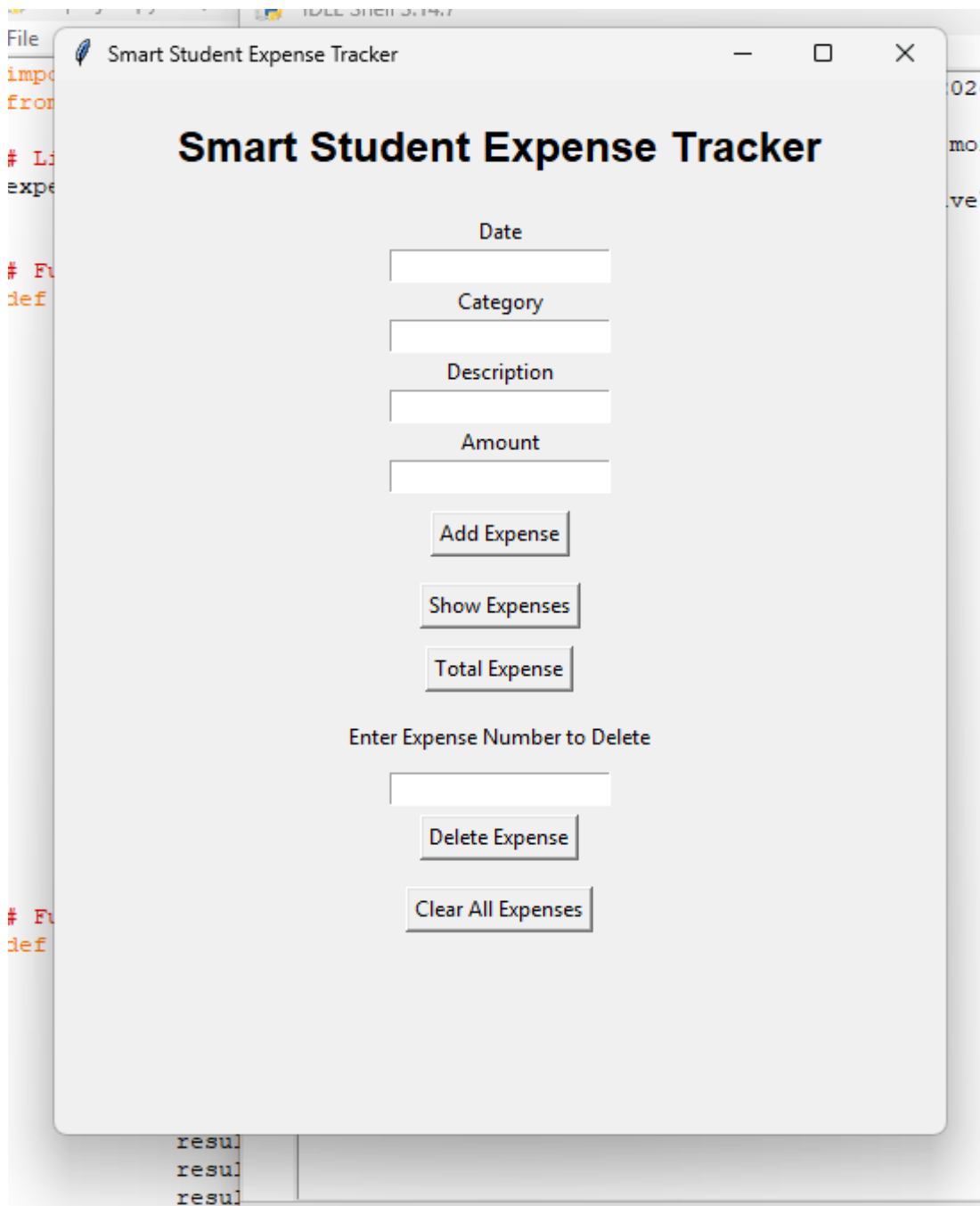
```
■■■ main.py
■■■ user.py
■■■ expenses.py
■■■ validation.py
■■■ storage.py
■■■ analysis.py
■■■ report.py
■■■ config.py
```

```
■■■ data/expenses.csv
■■■ tests/test_analysis.py
■■■ tests/test_validation.py
■■■ README.md
■■■ statement.md
■■■ requirements.txt
```

15. Screenshots / Results

The final submission should include screenshots of the running application. Recommended screenshots are:

- Main menu



- Adding a new expense

Smart Student Expense Tracker

Smart Student Expense Tracker

Date
23/09/2026

Category
PURCHASE

Description
BOOKS

Amount
1000

Add Expense

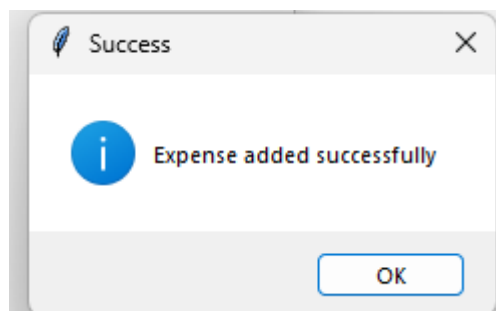
Show Expenses

Total Expense

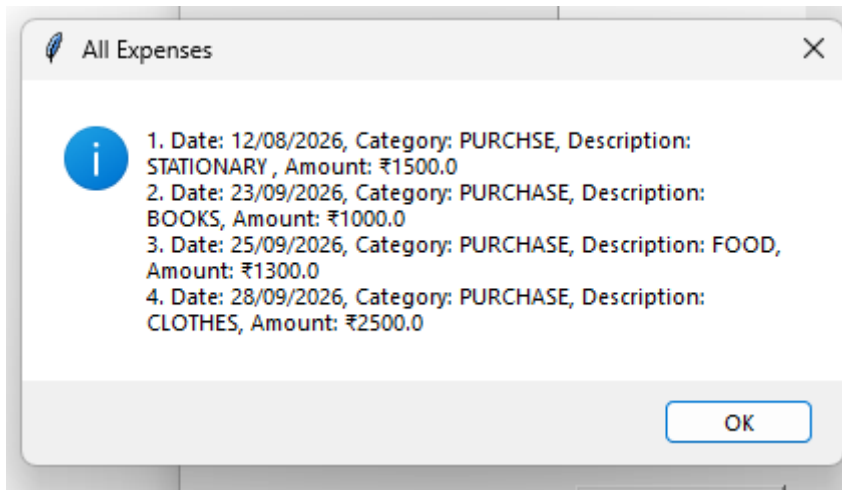
Enter Expense Number to Delete

Delete Expense

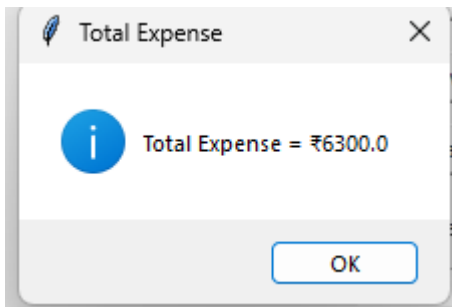
Clear All Expenses



- Viewing expense records



- Expense summary



16. Testing Approach

Testing combines unit tests and manual validation. The included unittest files verify calculation logic and category configuration.

Suggested test cases:

Test Case	Input / Condition	Expected Result
TC1	Valid expense amount	Expense is accepted.
TC2	Amount ≤ 0	Validation error is shown.
TC3	Invalid date	User is asked to enter a valid date.
TC4	Invalid category	User is asked to select a valid category.
TC5	Unknown delete ID	No record is deleted.
TC6	Multiple expenses	Totals and category values are calculated correctly.
TC7	Unit tests	All included tests pass.

17. Challenges Faced

- Designing a clean modular structure for a small application.
- Handling incorrect user input without crashing the program.

- Maintaining consistent CSV records when adding and deleting data.
- Converting stored string values into numeric amounts for calculations.
- Creating tests that verify the most important calculation logic.

18. Learnings & Key Takeaways

- Using functions to divide a program into reusable components.
- Using modules to organize source code.
- Working with lists and dictionaries.
- Reading and writing CSV files.
- Using exception handling for robust input processing.
- Applying basic software architecture and documentation.
- Writing and running unit tests.
- Using Git/GitHub for project version control.

19. Future Enhancements

- Develop a graphical interface using Tkinter.
- Replace CSV with SQLite for larger datasets.
- Add monthly budget limits and alerts.
- Add charts for visual spending analysis.
- Support PDF and Excel report export.
- Add authentication and multiple user profiles.

20. References

- Python documentation and standard library concepts.
- Course notes on Python programming, functions, modules, file handling and exception handling.
- VITyarthi – Build Your Own Project instructions supplied for the project.

Appendix: How to Run

1. Install Python 3.x.

2. Open the project folder in a Python editor or terminal.
3. Run **python main.py**.
4. Follow the menu prompts.
5. Run **python -m unittest discover -s tests -p "test_*.py" -v** to execute tests.